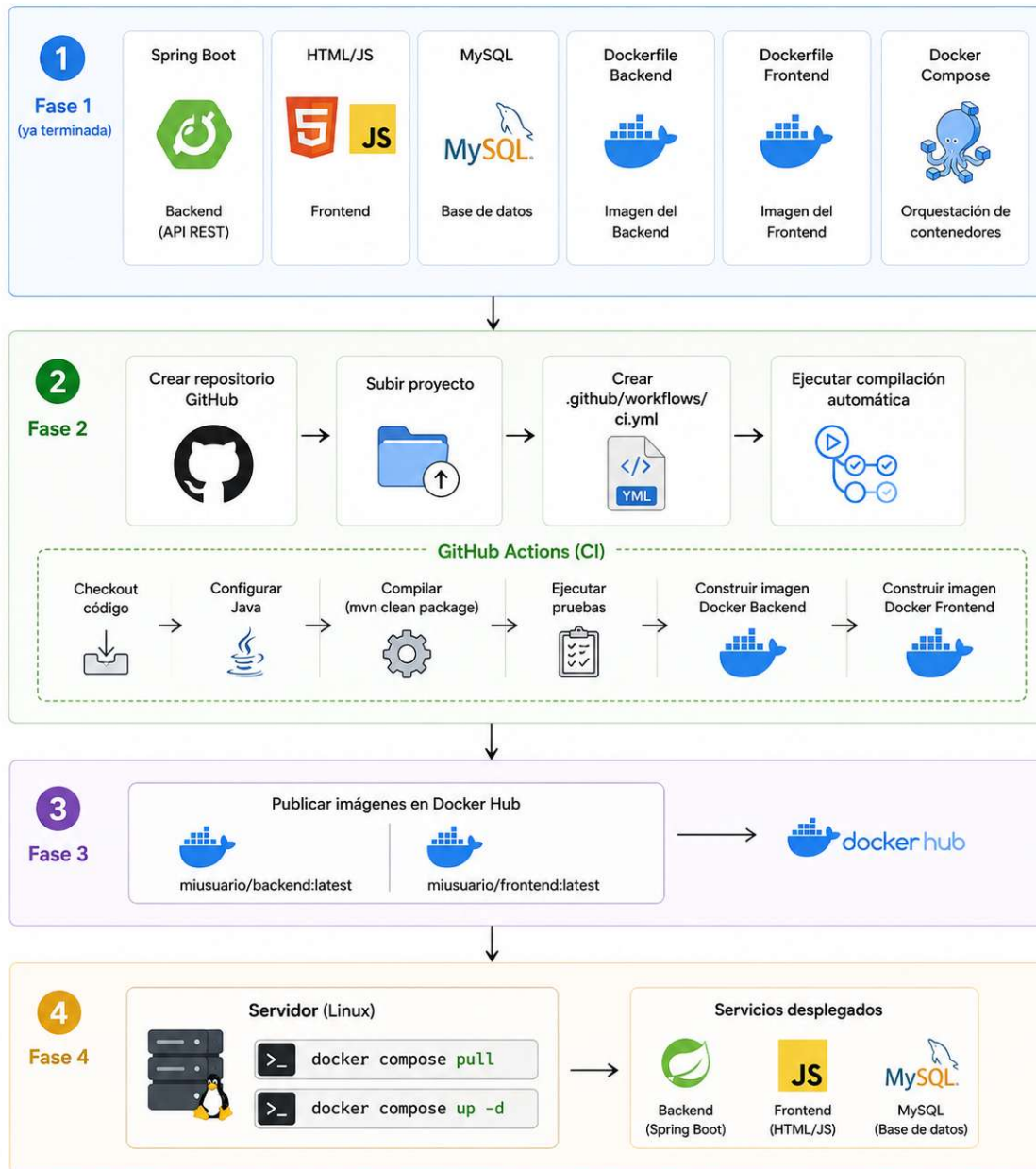


Docker Compose - CI - CD - Práctica

Práctica Guiada:



Primera Parte: Dockerfile – Docker Compose

Estructura de Proyecto para 15B:

Backend: SprinBoot - > 8090:8080 http://localhost:8090

Frontend: Html5, JS-> 8081:80-> servidor Nginx -> http://localhost:8082

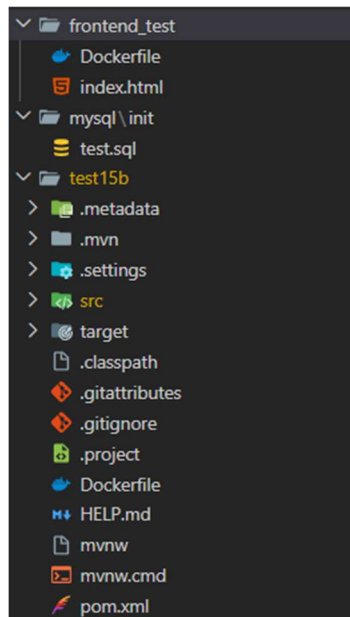
Base de Datos: MySQL-> puerto 3307 -> 3306

Estructura de Proyecto para 15A:

Backend: SprinBoot - > 8091:8080 http://localhost:8090

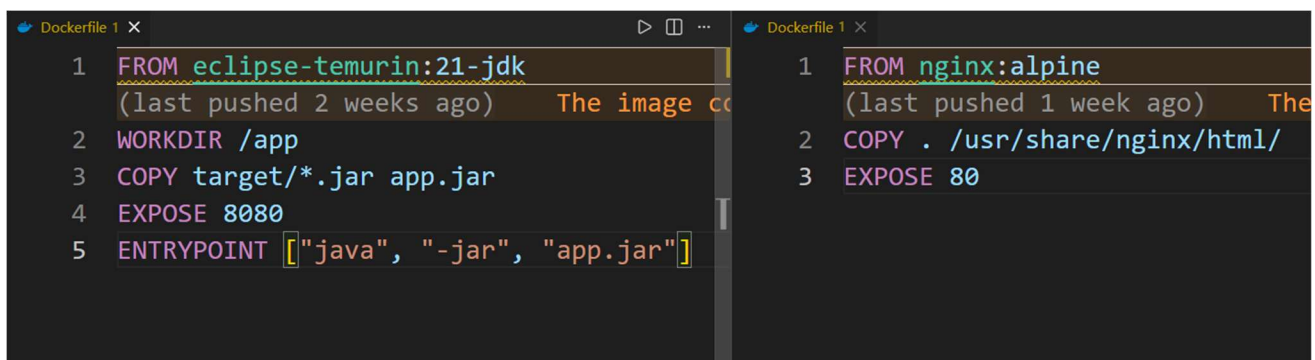
Frontend: Html5, JS-> 8082:80-> servidor Nginx -> http://localhost:8082

Base de Datos: MySQL-> puerto 3308 -> 3306

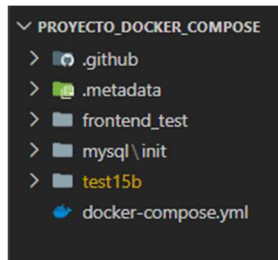


Integrar Docker Compose:

1. Crear Dockerfile del Backend y en el Frontend, como se muestra en la imagen y copiamos:



2. Ahora en la carpeta raíz del proyecto creamos: docker-compose.yml



3. Creamos los servicios en el **docker-compose.yml** – SE AGREGÓ LÍNEA 20 OJO

```
docker-compose.yml
  Run All Services
1  services:
  Run Service
2    mysql-db:
3      image: mysql:8.0
4      container_name: mysql-db
5      environment:
6        MYSQL_ROOT_PASSWORD: 1234
7        MYSQL_DATABASE: test
8      ports:
9        - "3307:3306"
10     volumes:
11       - ./mysql/init:/docker-entrypoint-initdb.d
12
  Run Service
13    backend:
14      build: ./test15b
15      container_name: backend-spring
16      ports:
17        - "8090:8080"
18      depends_on:
19        - mysql-db
20      restart: on-failure
21      environment:
22        SPRING_DATASOURCE_URL: jdbc:mysql://mysql-db:3306/test
23        SPRING_DATASOURCE_USERNAME: root
24        SPRING_DATASOURCE_PASSWORD: 1234
  Run Service
25    frontend:
26      build: ./frontend_test
27      container_name: frontend-html
28      ports:
29        - "8081:80"
30      depends_on:
31        - backend
```

4. Vamos a la consola y hacemos: **docker compose up --build**
5. Si nos da error en el back entramos a la carpeta Backend y ejecutamos `.\mvnw.cmd clean package -DskipTests`

docker compose down

docker compose build --no-cache

docker compose up -d

6. Verificamos en la terminal que los dos contenedores estén arriba: docker ps

```
PS C:\Users\ke12m\Desktop\proyecto_docker_compose> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5e0b97f061d7	proyecto_docker_compose-backend	"java -jar app.jar"	2 minutes ago	Up 2 minutes	0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp	backend-spring
83b2ddab5a53	mysql:8.0	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes	0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp	mysql-db

7. Abrimos Docker Desktop:

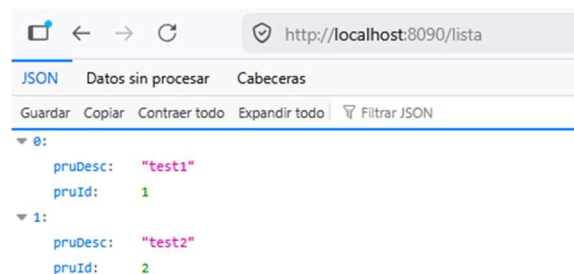
●	proyecto_docke	-	-	-	1.08%	691.9MB / 15.3G	8.83%	■
●	mysql-db	83b2ddab5a53	mysql:8.0	3307:3306 ↗	0.8%	379.1MB / 7.65G	4.84%	■
●	backend-spri	5e0b97f061d7	proyecto_docker_compos	8090:8080 ↗	0.28%	312.8MB / 7.65G	3.99%	■

8. Probamos en el navegador los endPoints:

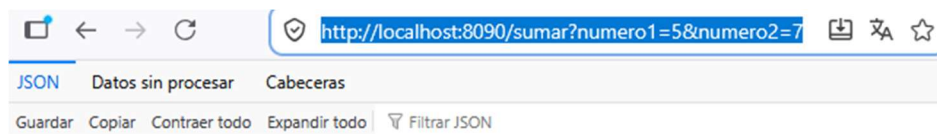
<http://localhost:8090/saludar>



<http://localhost:8090/lista>



<http://localhost:8090/sumar?numero1=5&numero2=7>



12

9. Agregamos el frontend en el archivo docker-compose.yml:

```

24     frontend:
25         build: ./frontend_test
26         container_name: frontend-html
27         ports:
28             - "8081:80"
29     depends_on:
30         - backend

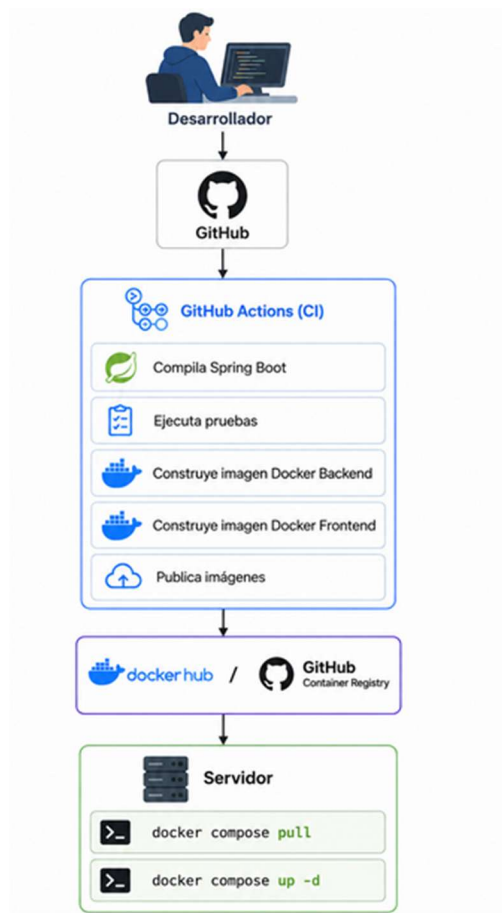
```

10. Ejecutamos: **docker compose up --build**

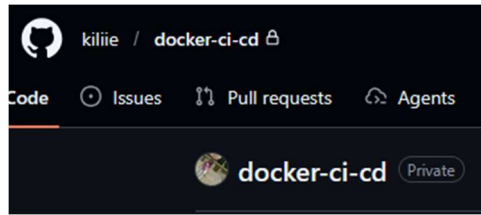
11. Probamos:



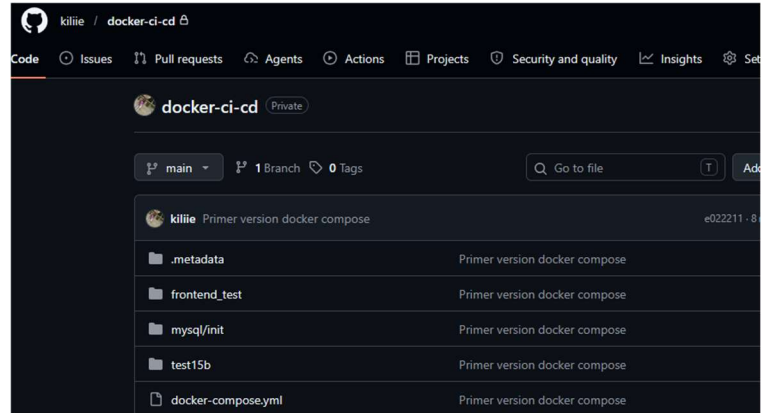
Segunda Parte: CI



1. Crear un repositorio en GitHub:

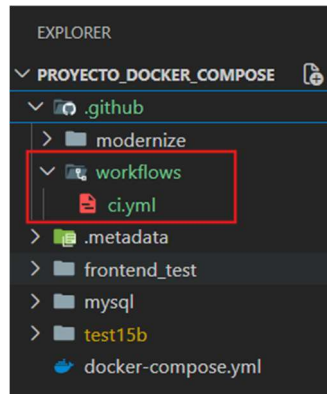


2. Inicializamos el repo: git init
3. Agregamos archivos: git add .
4. Primera versión: git commit -m "Primer version docker compose"
5. Conecto el repo: git remote add origin <https://github.com/xxxxx/docker-ci-cd.git>
6. git branch -M main
7. git push -u origin main



8. Ahora en el proyecto, dentro del directorio de .github crear:

Directorio: workflows y dentro un archivo ci.yml



9. Abrimos el archivo ci.yml y escribimos el siguiente código:

```
.github > workflows > ci.yml
1  name: CI Spring Boot Docker
2  on:
3    push:
4      branches:
5        - main
6  jobs:
7    build-backend:
8      runs-on: ubuntu-latest
9      defaults:
10       run:
11         working-directory: test15b
12     steps:
13       - name: Descargar código
14         uses: actions/checkout@v4
15       - name: Instalar Java 21
16         uses: actions/setup-java@v4
17         with:
18           distribution: temurin
19           java-version: 21
20       - name: Compilar proyecto
21         run: mvn clean package -DskipTests
22       - name: Verificar JAR generado
23         run: ls -la target
24       - name: Construir imagen Docker
25         run: docker build -t backend-springboot .
```

Explicación del **ci.yml**-> lo que se está escribiendo es **Github Actions Workflow o Workflow**

Cada bloque del archivo tiene su propósito:

Elemento	Significado
name	Nombre del workflow
on	Evento que lo dispara

jobs	Tareas que ejecutará
build-backend	Nombre del job
runs-on	Sistema operativo donde corre
steps	Pasos del job
uses	Usa una acción predefinida
run	Ejecuta comandos Linux
with	Parámetros de una acción

Veamos:

```
on:
  push:
    branches:
      - main
```

esto significa que cuando alguien haga push a main, ejecuta este workflow.

```
- name: Instalar Java 21
  uses: actions/setup-java@v4
```

Descarga e instala Java 21 en el servidor temporal de GitHub.

10. Antes de hacer push, vamos a entender que va a pasar:



11. Guardar el archivo y hacer: git add .

12. Agregamos el commit del cambio: git commit -m "Agregar workflow CI"

13. Y: git push

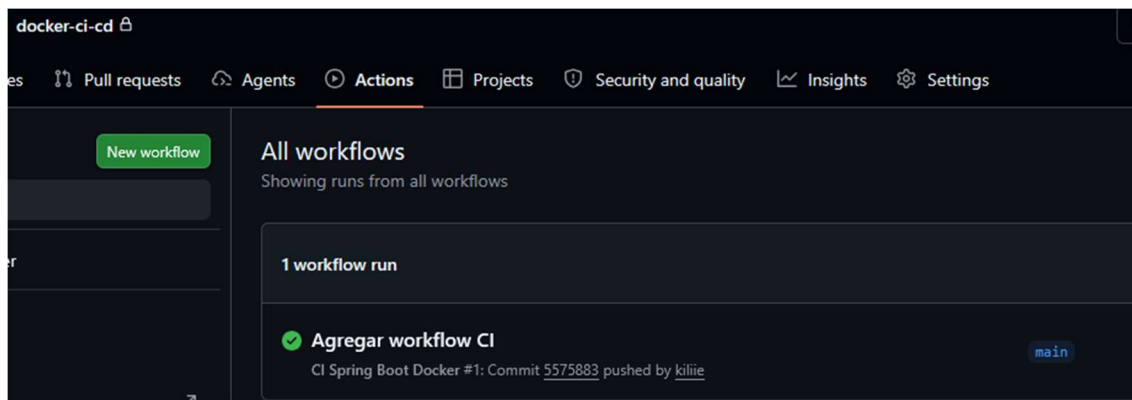

```

kel2m\Desktop\proyecto_docker_compose> git add .
kel2m\Desktop\proyecto_docker_compose> git commit -m "Agregar workflow CI"
[master 100644] Agregar workflow CI
 3 files changed, 33 insertions(+)
 create mode 100644 .github/workflows/ci.yml
kel2m\Desktop\proyecto_docker_compose> git push
Counting objects: 6, done.
Compressing objects: 100% (6/6), done.
Writing objects to repository: 100% (6/6), done.
Total 6 objects, 658 bytes | 658.00 KiB/s, done.
Creating deltas: 100% (1/1), completed with 1 local object.
Pushing to 'github.com:kiliie/docker-ci-cd.git'
Successfully pushed to 'github.com:kiliie/docker-ci-cd.git'
5575883 main -> main

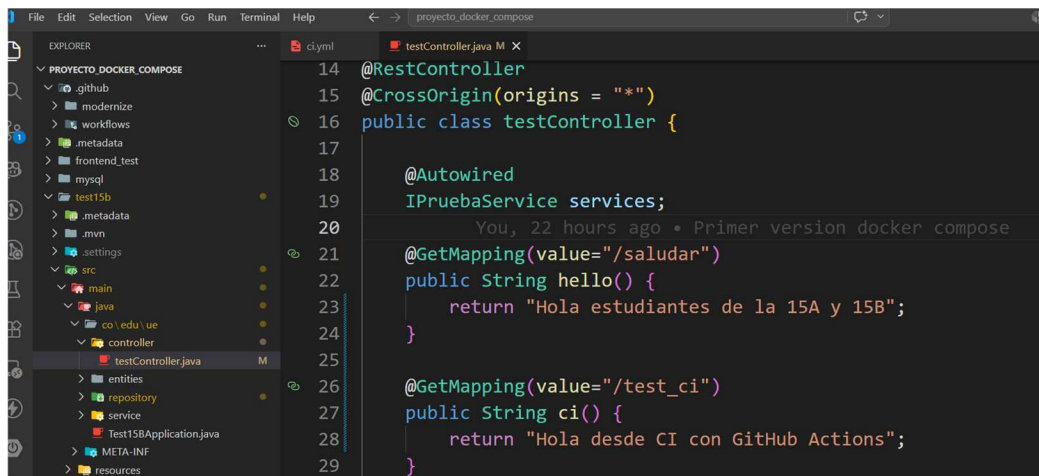
```

14. Ir a GitHub y en el repositorio revisar la pestaña **Actions**: si la ejecución salió en verde esto significa que GitHub:

1. Descargó tu código
2. Creó una máquina Ubuntu temporal
3. Instaló Java 21
4. Ejecutó Maven
5. Generó el .jar
6. Construyó la imagen Docker



15. Probemos, en la aplicación de Spring-Boot agregar un endPoint y modificar el de saludo:



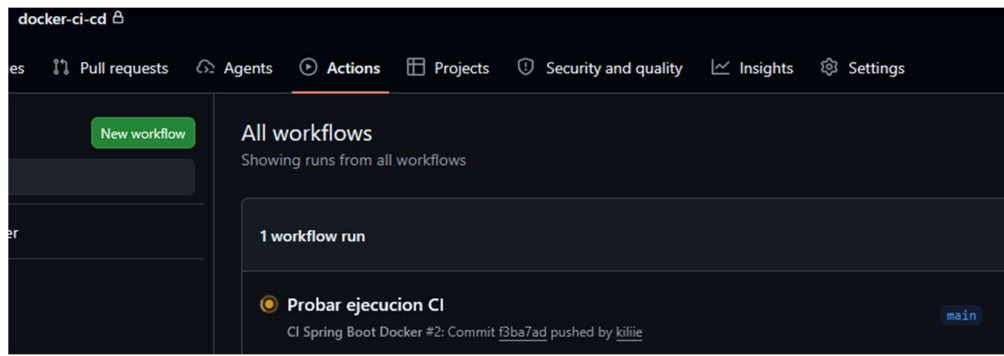
16. Hacemos git add . -> git commit -m "Probar ejecucion CI" -> git push

```

/github.com/kiliie/docker-ci-cd.git
..5575883 main -> main
s\kel2m\Desktop\proyecto_docker_compose> git add .
s\kel2m\Desktop\proyecto_docker_compose> git commit -m "Probar ejecucion CI"
[7ad] Probar ejecucion CI
changed, 6 insertions(+), 1 deletion(-)
s\kel2m\Desktop\proyecto_docker_compose> git push
g objects: 21, done.
objects: 100% (21/21), done.
compression using up to 8 threads
g objects: 100% (8/8), done.
objects: 100% (11/11), 877 bytes | 877.00 KiB/s, done.
delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
solving deltas: 100% (4/4), completed with 4 local objects.
/github.com/kiliie/docker-ci-cd.git
..f3ba7ad main -> main

```

17. Aquí vemos que se creó una nueva ejecución y está en proceso



18. Aquí ya terminó: ya en verde la CI comprobó que el proyecto sigue compilando



Nota: aquí ya tenemos integración continua CI